Configuration and metadata

Website settings live in `calepin.toml` at the root of your website source directory. Use `--config <path>` only when the config file lives somewhere else.

Basic settings

A small site can start with:

```
# Site name shown by bundled themes and page metadata.
# Default: none.
title = "My Site"

# Short site description for metadata and previews.
# Default: none.
description = "A static website built from Typst documents."

# Public site URL, used for sitemap.xml and canonical URLs.
# Default: none.
base_url = "https://example.com"

# Theme: "calepin", "academic", a local theme directory, or false for raw output.
# Default: "calepin".
theme = "academic"

# Render .pdf files for every page.
# Default: false.
pdf = false

# Minify generated HTML, including inline CSS and JavaScript.
# Default: false.
minify = true

# Static search engine. Set to "pagefind" to generate a search index.
# Default: none.
search = "pagefind"

# Logo image path for the top bar.
# Default: none.
logo = "assets/logo.svg"

# Alternative text for the logo image. Defaults to `title` when omitted.
# Default: none.
logo_alt = "My Site"

# Browser favicon path. Omit this to use Calepin's generated default.
# Default: ".calepin/favicon.svg".
favicon = "assets/favicon.ico"

# HTML syntax highlighting themes. Paths are relative to calepin.toml.
# Default: built-in Calepin light and dark themes.
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Use `--format html` for a one-time HTML-only build, regardless of `pdf`. Use `--minify` to minify HTML for a single build without changing `calepin.toml`.

Set `search = "pagefind"` to add static search to bundled website themes. *Calepin* writes the Pagefind search bundle to `pagefind/` after rendering the site and links the bundled themes to the generated component script and stylesheet. Bundled themes mark the main page content with `data-pagefind-body`, so navigation, toolbars, and footers are excluded from the search index.

Paths in `calepin.toml` are interpreted from the website source directory: the directory that contains the config file, unless you pass an explicit `--config`. This includes `logo`, `favicon`, page `target` values ending in `.typ`, page `glob` patterns, page and static `include` and `exclude` patterns, and local theme paths. During the build, *Calepin* rewrites internal files and generated assets as page-relative URLs, so links and images continue to work from nested pages. Literal URLs such as `https://example.com` are left unchanged.

Paths inside `.typ` files follow Typst path rules, not `calepin.toml` rules. In website builds, use root-relative Typst paths for shared website assets, such as `#image("/assets/diagram.svg")`; the leading `/` points at the website source directory, so the same source works from pages in subdirectories. Avoid bare relative paths such as `#image("assets/diagram.svg")` for shared assets in nested pages. If no favicon is set, *Calepin* writes a small default to `.calepin/favicon.svg`; if no logo is set, bundled themes use `title` as the site name.

Syntax highlighting

Configure HTML syntax highlighting with full TextMate `.tmTheme` files:

```
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Paths are resolved relative to `calepin.toml`. The light theme is used for light browser mode, and the dark theme is used for dark browser mode.

Paged output is different: PDF, SVG, and PNG are rendered directly by Typst, so they use Typst's standard `raw` highlighting. If you want a fixed paged syntax theme, customize it in Typst:

```
#let paper-code-theme = read("themes/syntax/paper.tmTheme", encoding: none)

#show raw.where(block: true): it => {
  raw(it.text, block: true, lang: it.lang, theme: paper-code-theme)
}
```

For HTML, *Calepin* keeps syntax tokenization consistent with Typst. Typst still reads Sublime syntax definitions (`.sublime-syntax`) and highlights the code; *Calepin* then maps the generated HTML colors to CSS classes so the final colors can depend on light or dark mode.

The HTML mapping uses the TextMate settings that affect rendered spans: `foreground`, `background`, and `fontStyle` values such as `bold`, `italic`, and `underline`. Use light and dark themes with similar scope rules for the most predictable result. If a scope exists in one theme but not the other, *Calepin* falls back to that theme's global foreground.

Robots.txt

Calepin writes `robots.txt` by default:

```
User-agent: *
Allow: /
Sitemap: https://example.com/sitemap.xml
```

The `Sitemap:` line is included when `base_url` is set. To disable generation:

```toml
robots = false
```

or:

```toml
[robots]
enabled = false
```

To override the generated file, create `templates/robots.txt` in the website source directory. The template uses MiniJinja syntax and receives `config` and `sitemap_url`:

```
User-agent: *
Disallow: /drafts/
{% if sitemap_url %}Sitemap: {{ sitemap_url }}{% endif %}
```

Files under `templates/` can be included or extended from the robots template:

```
{% extends "base.txt" %}
{% block body %}
User-agent: *
Allow: /
{% endblock %}
```

Static files

Use `[static]` for files that should be copied to the built website without being rendered as Typst pages:

```toml
[static]
include = [
  "assets/**",
  "CNAME",
  "downloads/**",
]
exclude = [
  "assets/drafts/**",
]
```

Each included file keeps its source-relative path in the output directory: `assets/logo.svg` becomes `assets/logo.svg`, `CNAME` becomes `CNAME`, and `downloads/manual.pdf` becomes `downloads/manual.pdf`. `include` entries can be files, directories, or glob patterns. `exclude` patterns are applied after includes. Paths must stay inside the website source directory.

Navigation

Side bar

The sidebar is the main navigation for documentation-style sites. Configure it with `[sidebar]`; a section can list pages one by one:

```toml
[sidebar]

[[sidebar.section]]
title = "Guide"

  [[sidebar.section.item]]
  target = "install.typ"
```

or include several pages with a glob:

```toml
[[sidebar.section.item]]
glob = "guide/*.typ"
```

Use `target` for one source page and `glob` for a list of source pages. Sidebar entries always point to Typst source files, not rendered `.html` files. *Calepin* resolves those pages and writes the right `.html` links in the generated site.

The sidebar label comes from the page source, not from `calepin.toml`. Put the label in the page's website metadata:

```
#set document(title: [Install])
#metadata((title: "Install")) <website-metadata>

#title()
```

If a page has no `website-metadata.title`, *Calepin* falls back to the document title, then the filename stem. This keeps multilingual sidebars in one place: each translated page carries its own translated title.

If you do not configure a sidebar, *Calepin* builds one from `.typ` files in the source directory. Hidden files are skipped.

Titled sections are foldable: each page loads with the section that contains it open and the others folded, and readers can open more sections by hand. To keep every section expanded instead, disable folding:

```toml
[sidebar]
fold = false
```

Sidebar icons are configured separately from labels:

```toml
[[sidebar.section.item]]
target = "install.typ"
icon = "lucide:download"
```

If the prefix is omitted, *Calepin* uses `lucide`, so `icon = "home"` means `icon = "lucide:home"`. Icons are downloaded during the build and cached under `.calepin/icons/`.

Icon prefixes are Iconify collection names. Search available icons in the Iconify icon sets browser. Common prefixes include `lucide`, `simple-icons`, `tabler`, `heroicons`, `material-symbols`, `carbon`, `ph`, and `bi`.

Top bar

Use `[navbar]` for a small top navigation bar. External links, such as a GitHub repository, are regular navbar items:

```toml
[navbar]

[[navbar.item]]
position = "left"
target = "index.typ"
label = "Home"

[[navbar.item]]
position = "right"
target = "https://github.com/user/repo"
label = "{icon:github} GitHub"
```

Navbar items use `target` or `glob`. Use a `.typ` `target` or `glob` for internal source pages; use any other `target` for external links or a literal already-rendered URL. `position` can be `left`, `center`, or `right`.

Include or exclude pages

Use `[pages]` for Typst pages that should be built but should not appear in navigation, such as blog posts or legal pages:

```toml
[pages]
include = ["blog/*.typ", "legal/privacy.typ"]
exclude = ["drafts/**"]
```

Put the page title in the document and keep website metadata for fields used by listings, routing, and output options:

```typst
#set document(title: [First post])

#metadata((
  date: "2026-06-10",
  tags: ("release", "website"),
)) <website-metadata>

#title()
```

`index.typ` and `404.typ` are always built when present. If `404.typ` exists, *Calepin* writes `404.html`; if PDF rendering is enabled for that page, it also writes `404.pdf`.

Pages metadata

Add arbitrary page metadata with Typst's `#metadata` function and the `<website-metadata>` label. *Calepin* reads this dictionary while building the site and attaches it to the page entry returned by `calepin.pages()`.

```typst
#set document(title: [First post])

#metadata((
  date: "2026-06-10",
  tags: ("release", "website"),
  author: "Ada Lovelace",
  summary: "A short release note for the new website.",
  draft: false,
)) <website-metadata>

#title()
```

Use `calepin.pages()` to get structured information about every built page, including its metadata, and process it with normal Typst functions and methods. This is useful for lists, indexes, feeds, publication pages, course pages, and any page that needs to organize other pages in the site.

```typst
#import "/.calepin/calepin.typ" as calepin

#let posts = calepin.pages()
  .filter(p => p.path.starts-with("blog/"))
  .filter(p => not p.meta.at("draft", default: false))
  .sorted(key: p => p.meta.at("date", default: ""))
  .rev()
```

```
#for post in posts [
  - #link(post.href)[#post.title] \
    #post.meta.at("summary", default: [])
]
```

`calepin.pages()` returns one dictionary per built page, excluding `404.typ`. *Calepin* creates these entry fields:

- `path`: source path relative to the website root
- `href`: rendered HTML path relative to the current page
- `title`: resolved page title
- `language`: language code, or `none` when languages are not configured
- `translation_key`: resolved key used to connect translated pages
- `translations`: matching pages in other languages, or an empty dictionary
- `pdf`: PDF path, or `none` when the page has no PDF output
- `meta`: the page's `<website-metadata>` dictionary, or an empty dictionary

Only `meta` comes from the page's `#metadata` value. *Calepin* interprets a few optional metadata keys: `title`, `pdf`, `translation_key`, `slug`, and `url`. All other keys are left untouched for your own Typst code.

There is no required schema for custom metadata. Common keys include `date`, `tags`, `author`, `authors`, `category`, `venue`, `summary`, and `draft`, but you can use any key your page list or template expects. Since `calepin.pages()` returns a Typst array of dictionaries, you can use Typst functions and methods such as `filter`, `map`, `sorted`, `rev`, `at`, and `contains` to select and format the pages you need.

Multilingual sites

Configure languages with one content directory per language:

```
default_language = "en"

[languages.en]
label = "English"
content_dir = "."

[languages.fr]
label = "Français"
content_dir = "fr"
```

With this layout, `about.typ` and `fr/about.typ` are treated as translations of the same page. The default language keeps root URLs like `about.html`; other languages use their language code as a URL prefix, such as `fr/about.html`.

Use page metadata when translations move or need different slugs:

```
#set document(title: [À propos])

#metadata((
  translation_key: "about",
  slug: "a-propos",
)) <website-metadata>

#title()
```

When more than one language is configured, the bundled themes show a language picker. Local navigation links are shown only for the current language; external links stay global.